

Datentyp `String` in Java

Merkmal	Beschreibung
Immutabilität	Ein <code>String</code> in Java ist unveränderlich (immutable). Das bedeutet, dass jede Änderung an einem <code>String</code> tatsächlich ein neues <code>String</code> -Objekt erzeugt.
Speicherverwaltung	<code>String</code> -Objekte werden häufig durch den String-Pool verwaltet, was Speicher spart, indem wiederholte Instanzen des gleichen Strings nicht im Speicher gehalten werden.
Speicherbedarf	<code>String</code> verwendet eine UTF-16-codierte Zeichencodierung, die zwei Bytes pro Zeichen benötigt. Dies beeinflusst den Speicherbedarf im Vergleich zu anderen Zeichentypen.
Thread-Sicherheit	Aufgrund seiner Unveränderlichkeit ist der <code>String</code> in Java von Natur aus thread-sicher.
Vergleich von Strings	Zwei <code>String</code> -Objekte sollten nicht mit <code>==</code> verglichen werden, sondern mit der <code>equals()</code> -Methode, da <code>==</code> nur die Referenzen vergleicht, nicht den Inhalt.
Performance	Da <code>String</code> unveränderlich ist, sind String-Verkettungen und -Modifikationen, insbesondere in Schleifen, teurer. In solchen Fällen sollte <code>StringBuilder</code> oder <code>StringBuffer</code> verwendet werden.

Methoden

Methode	Beschreibung
<code>charAt(int index)</code>	Gibt das Zeichen an der angegebenen Position im String zurück.
<code>length()</code>	Gibt die Länge des Strings zurück, d. h. die Anzahl der Zeichen im String.
<code>substring(int beginIndex)</code>	Gibt einen neuen String zurück, der mit dem Zeichen an der angegebenen <code>beginIndex</code> -Position beginnt und bis zum Ende des Strings reicht.
<code>substring(int beginIndex, int endIndex)</code>	Gibt einen neuen String zurück, der die Zeichen im Bereich von <code>beginIndex</code> bis <code>endIndex - 1</code> enthält.
<code>equals(Object obj)</code>	Vergleicht den String mit dem angegebenen Objekt auf Gleichheit. Gibt <code>true</code> zurück, wenn der String mit dem Objekthinhalt übereinstimmt.
<code>equalsIgnoreCase(String anotherString)</code>	Vergleicht den String mit einem anderen String ohne Berücksichtigung der Groß- und Kleinschreibung.

Methode	Beschreibung
<code>compareTo(String anotherString)</code>	Vergleicht den aktuellen String mit einem anderen String lexikografisch und gibt einen Wert zurück: negativ, wenn der aktuelle String lexikografisch kleiner ist, positiv, wenn er größer ist, oder 0, wenn beide gleich sind.
<code>indexOf(int ch)</code>	Gibt die Position des ersten Vorkommens des angegebenen Zeichens im String zurück oder <code>-1</code> , wenn das Zeichen nicht gefunden wird.
<code>lastIndexOf(int ch)</code>	Gibt die Position des letzten Vorkommens des angegebenen Zeichens im String zurück oder <code>-1</code> , wenn das Zeichen nicht gefunden wird.
<code>contains(CharSequence sequence)</code>	Überprüft, ob der String die angegebene Zeichenfolge enthält und gibt <code>true</code> zurück, wenn dies der Fall ist.
<code>replace(CharSequence target, CharSequence replacement)</code>	Ersetzt alle Vorkommen der angegebenen Zeichenfolge im String durch eine neue Zeichenfolge.
<code>split(String regex)</code>	Teilt den String anhand des angegebenen regulären Ausdrucks und gibt ein Array von Strings zurück.
<code>toLowerCase()</code>	Gibt einen neuen String zurück, in dem alle Zeichen des aktuellen Strings in Kleinbuchstaben umgewandelt sind.
<code>toUpperCase()</code>	Gibt einen neuen String zurück, in dem alle Zeichen des aktuellen Strings in Großbuchstaben umgewandelt sind.
<code>trim()</code>	Gibt einen neuen String zurück, bei dem führende und nachfolgende Leerzeichen entfernt wurden.
<code>startsWith(String prefix)</code>	Überprüft, ob der String mit der angegebenen Zeichenfolge beginnt. Gibt <code>true</code> zurück, wenn dies der Fall ist.
<code>endsWith(String suffix)</code>	Überprüft, ob der String mit der angegebenen Zeichenfolge endet. Gibt <code>true</code> zurück, wenn dies der Fall ist.
<code>toCharArray()</code>	Gibt ein Char-Array zurück, das den Inhalt des Strings darstellt.